

μ -GALOIS: Triplet Problem, computation details, pseudocodes

Thomas Ceresa

Supervisor: Gianmaria Silvello

Academic: Chao Zhang, Angela Bonifati

end April 2026

This paper introduces the first computation details of μ -Galois¹. We restrict ourselves, in a first step, to the following problems (particularly **T1**, **T2**, **T3**), without conditions' aspects:

($\mathcal{S}$) **Triplet patterns.** RDF triple patterns admit several shapes, depending on which of the subject, predicate and object are variables. Consider:

```
T1  ?x st:isFriendWith st:Mike
T2  ?x st:isFriendWith ?y
T3  st:Mike st:isFriendWith ?y
T4  ?x ?y st:Mike
T5  ?x ?y ?z
```

1. Prompts

Figure 1 presents the prompt syntax for the three physical strategies of LLMTRIPLETSCAN, together with the confidence prompt used to route between SEEDCRANK and KEYCRANK. Within KEYCRANK, the direction is chosen deterministically from seed sizes (cf. Algorithm 4). Concrete instantiations on patterns T1, T2 and T3 are given alongside each strategy.

Following the declarative prompt design of LOTUS [?], each prompt is structured around two explicit sections: a **Context** block describing what is already known, and a **Task** block stating what the LLM must produce. Common behavioural rules (no hallucination, JSON-only output, return empty list if uncertain) are factored into a shared *system prompt* sent once per call, keeping individual prompts concise and token-efficient.

(Common) System Prompt (sent once per LLM call, shared across all strategies):

You are an expert RDF knowledge graph assistant. Your job is to retrieve factual RDF triples from your knowledge with high precision. You will be given a *Context* describing what is already known, and a *Task* to perform. Never invent, speculate, or hallucinate entities or relationships. Only assert triples you are certain about. If uncertain or no triples exist, return `{"triples": []}`. Always respond in valid JSON using this schema: `{"triples": [{"s": "...", "p": "...", "o": "..."}]}`. Never add explanations or text outside the JSON structure.

¹The precedent paper of the previous week: <https://www.overleaf.com/read/hjtzqjzcszsz>

(i) **TableScan** — Prompt:

Context: No prior information is available about `predicateURI`.

Task: Given the predicate `predicateURI`, list all known triples (`subjectTerm`, `predicateURI`, `objectTerm`) that factually hold. If you are unsure or no triples exist, return an empty list.

(ii) **SeedCrank** — Prompt:

Context: The following values are already known:

(-- `subjectTerm` is one of: {`seedsS`})

(-- `objectTerm` is one of: {`seeds0`})

Task: List all triples (`subjectTerm`, `predicateURI`, `objectTerm`) that factually hold, knowing the constraints above. Return only factual values. Do not introduce entities outside the sets provided.

Example — SeedCrank on T2 (`?x :isFriendWith ?y`), $v(?x) = \{\text{Mike, Dustin, Doc}\}$, $v(?y) = \{\text{Demogorgon, Papa}\}$:

Context: The following values are already known:

— `?x` is one of: Doc, Dustin, Mike

— `?y` is one of: Demogorgon, Papa

Task: List all triples (`?x`, `:isFriendWith`, `?y`) that factually hold ...

(iii) **KeyCrank** — Left-to-right (one prompt per $k \in v(s)$):

Context: The subject is fixed: `seedValue`. The predicate is `predicateURI`.

(-- `objectTerm` is one of: {`seeds0`})

Task: List all triples (`seedValue`, `predicateURI`, `objectTerm`) that factually hold. Return only factual values.

Example — KeyCrank L→R on T2, seed value Mike, $v(?y) = \{\text{Demogorgon, Papa}\}$:

Context: The subject is fixed: Mike. The predicate is `:isFriendWith`.

— `?y` is one of: Demogorgon, Papa

Task: List all triples (Mike, `:isFriendWith`, `?y`) that factually hold ...

KeyCrank — Right-to-left (one prompt per $k \in v(t)$):

Context: The object is fixed: `seedValue`. The predicate is `predicateURI`.

(-- `subjectTerm` is one of: {`seedsS`})

Task: List all triples (`subjectTerm`, `predicateURI`, `seedValue`) that factually hold. Return only factual values.

Example — KeyCrank on T1 ($?x$:isFriendWith $st:Mike$), seed value Dustin: both sides are fully determined ($k = \text{Dustin}$, $o = st:Mike$); the algorithm uses the **Check** prompt instead.

Example — KeyCrank on T3 ($st:Mike$:isFriendWith $?y$), seed value Eleven: same situation ($s = st:Mike$, $k = \text{Eleven}$); Check prompt used.

(iv) Strategy Confidence (LLMConf) — Prompt (called once before choosing the strategy):

Context: The following values are already known:

(-- subjectTerm is one of: {seedsS})

(-- objectTerm is one of: {seedsO})

Task: Given the triple pattern (subjectTerm, predicateURI, objectTerm), how confident are you to list at once, in a single answer, all factual triples that hold? Respond only with a number in $[0, 1]$, where 1 means fully confident and 0 means unable. Do not add any comment.

If the returned confidence is $> \tau_P$, SEEDCRANK is used (one single prompt). Otherwise, KEYCRANK is used (safer, one prompt per seed value).

(Common) Iterative Prompt:

Context: The following triples have already been retrieved: {already_found}.

Task: List more triples if there are more. Do not repeat already retrieved values. If there are no more, return an empty list.

(Common) Check Prompt:

Context: We are verifying a single factual triple about the predicate predicateURI.

Task: Is this triple (subjectTerm, predicateURI, objectTerm) factually valid? Respond only with ‘yes’ or ‘no’. If you are uncertain, answer ‘no’.

Notation. predicateURI: the bound predicate URI, e.g. :isFriendWith. subjectTerm, objectTerm: either a variable name ($?x$) or a bound URI/literal ($st:Mike$). In SeedCrank, each constraint line is included only when the corresponding term is a variable with $v(?x) \neq \emptyset$. seedValue (KeyCrank): the single value from $v(s)$ or $v(t)$ fixed in this prompt instance. The iterated side is the smaller of $v(s)$ and $v(t)$; the opposite side may carry a seed constraint (cf. Algorithm 4). When both sides are fully determined (constant or single-value seed), KeyCrank falls back to the Check prompt (cf. Algorithm 4). τ_P : threshold for strategy routing.

Figure 1: Prompt syntax for the three physical strategies of LLMTRIPLETSCAN (i–iii), the confidence prompt used to route between SEEDCRANK and KEYCRANK (iv), and the common helper prompts. Text in italic is injected from the triple pattern $\langle s, p, o \rangle$ and the environment v . Lines between parentheses are included only when the corresponding condition applies. Strategy prompts return a list of full triples (s, p, o) ; the confidence prompt returns a single number in $[0, 1]$. Prompt structure follows the Context/Task declarative design of LOTUS [?].

1. Pseudo-code

LLMTRIPLETSCAN returns a relation T with three columns named after the terms of the pattern ($subjectTerm$, $predicateTerm$, $objectTerm$), and updates the environment v as a side-effect. Algorithm 1 dispatches between the three strategies, implemented in Algorithms 2–4.

Helpers. $seedsOf(x, v) = v(x)$ if $x \in \mathcal{V}$, else $\{x\}$. $parseTriples(r)$ parses the JSON response of the LLM. $updateEnv(v, T, s, o)$ sets $v(s) \leftarrow \pi_s(T)$ if $s \in \mathcal{V}$, and $v(o) \leftarrow \pi_o(T)$ if $o \in \mathcal{V}$. ctx is the conversational context (previous prompt/response pairs), used by TABLESCAN and SEEDCRANK to elicit new values without repetition.

Algorithm 1: LLMTRIPLETSCAN(s, p, o, v)

Input : triple pattern $\langle s, p, o \rangle$, environment v , threshold τ_P , language model LLM

Precondition: $p \in \mathcal{U}$ (predicate is a bound URI)

Output : table T with columns (s, p, o) ; environment v updated as a side-effect

```

1  $seedsS \leftarrow seedsOf(s, v)$ ;  $seedsO \leftarrow seedsOf(o, v)$ 
   // no seeds at all: fall back to TableScan
2 if  $seedsS = \emptyset$  and  $seedsO = \emptyset$  then
3   | return LLMTABLESCAN( $s, p, o, v$ )
4  $c \leftarrow \text{LLMCONF}(s, p, o, v)$  // Prompt (iv)
5 if  $c > \tau_P$  then
6   | return LLMSEEDCRANK( $s, p, o, v$ )
7 else
8   | return LLMKEYCRANK( $s, p, o, v$ )

```

Algorithm 2: LLMTABLESCAN(s, p, o, v) — inspired by Galois Alg. 1

Input : triple pattern $\langle s, p, o \rangle$, environment v , max iterations $maxIter$, language model LLM

Precondition: $seedsOf(s, v) = \emptyset$ and $seedsOf(o, v) = \emptyset$

Output : table T with columns (s, p, o) ; environment v updated as a side-effect

```
1  $i \leftarrow 0$ ;  $ctx \leftarrow []$ ;  $T \leftarrow \emptyset$ 
2 while  $i < maxIter$  do
3   if  $i = 0$  then
4      $prompt \leftarrow \text{genTableScanPrompt}(s, p, o)$  // Prompt (i)
5   else
6      $prompt \leftarrow \text{genIterativePrompt}(T)$  // Common
7    $r \leftarrow \text{LLM.request}(prompt, ctx)$ 
8    $T_{\text{new}} \leftarrow \text{parseTriples}(r)$ 
9   if  $T_{\text{new}} \subseteq T$  then
10    break // saturation
11    $ctx.add(prompt)$ ;  $ctx.add(r)$ 
12    $T \leftarrow T \cup T_{\text{new}}$ 
13    $i \leftarrow i + 1$ 
14  $\text{updateEnv}(v, T, s, o)$  // side-effect
15 return  $T$ 
```

Algorithm 3: LLMSEEDCRANK(s, p, o, v)

Input : triple pattern $\langle s, p, o \rangle$, environment v , max iterations $maxIter$, language model LLM

Precondition: at least one of s, o has a non-empty seed in v

Output : table T with columns (s, p, o) ; environment v updated as a side-effect

```
1  $i \leftarrow 0$ ;  $ctx \leftarrow []$ ;  $T \leftarrow \emptyset$ 
2 while  $i < maxIter$  do
3   if  $i = 0$  then
4      $prompt \leftarrow \text{genSeedCrankPrompt}(s, p, o, v)$  // Prompt (ii)
5   else
6      $prompt \leftarrow \text{genIterativePrompt}(T)$  // Common
7    $r \leftarrow \text{LLM.request}(prompt, ctx)$ 
8    $T_{\text{new}} \leftarrow \text{parseTriples}(r)$ 
9   if  $T_{\text{new}} \subseteq T$  then
10    break // saturation
11    $ctx.\text{add}(prompt)$ ;  $ctx.\text{add}(r)$ 
12    $T \leftarrow T \cup T_{\text{new}}$ 
13    $i \leftarrow i + 1$ 
14  $\text{updateEnv}(v, T, s, o)$  // side-effect
15 return  $T$ 
```

Algorithm 4: LLMKEYCRANK(s, p, o, v)

Input : triple pattern $\langle s, p, o \rangle$, environment v , language model LLM
Precondition: at least one of s, o has a non-empty seed in v
Output : table T with columns (s, p, o) ; environment v updated as a side-effect

```
1  $seedsS \leftarrow \text{seedsOf}(s, v); \quad seedsO \leftarrow \text{seedsOf}(o, v)$   
    $// \text{ iterate over the smaller seed set; tie-break L} \rightarrow \text{R}$   
2 if  $|seedsS| \leq |seedsO|$  then  
3    $dir \leftarrow \text{L} \rightarrow \text{R}; \quad iter \leftarrow seedsS; \quad other \leftarrow o; \quad otherSeeds \leftarrow seedsO$   
4 else  
5    $dir \leftarrow \text{R} \rightarrow \text{L}; \quad iter \leftarrow seedsO; \quad other \leftarrow s; \quad otherSeeds \leftarrow seedsS$   
6  $T \leftarrow \emptyset$   
7 foreach  $k \in iter$  (parallelizable) do  
    $// \text{ other side fully determined: verify via Check prompt}$   
8   if  $other \notin \mathcal{V}$  or  $|otherSeeds| = 1$  then  
9      $k' \leftarrow \text{unique element of } otherSeeds$   
10     $triple \leftarrow (k, p, k') \text{ if } dir = \text{L} \rightarrow \text{R}, \text{ else } (k', p, k)$   
11     $prompt \leftarrow \text{genCheckPrompt}(triple)$   $// \text{ Check prompt}$   
12     $r \leftarrow \text{LLM.request}(prompt)$   
13    if  $r = \text{"yes"}$  then  
14       $T \leftarrow T \cup \{triple\}$   
15  else  
16     $prompt \leftarrow \text{genKeyCrankPrompt}(s, p, o, v, k, dir)$   $// \text{ Prompt (iii)}$   
17     $r \leftarrow \text{LLM.request}(prompt)$   
18     $T \leftarrow T \cup \text{parseTriples}(r)$   
19  $\text{updateEnv}(v, T, s, o)$   $// \text{ side-effect}$   
20 return  $T$ 
```
